

Sistema de Simulación de Parque de Diversiones

Programación Concurrente 2025

Dante R. Giuliano | FAI - 2062

1 de diciembre de 2025

Índice

0.1. Objetivos del Sistema

- Simular el funcionamiento concurrente de un parque de diversiones.
- Gestionar el acceso controlado a recursos compartidos (atracciones).
- Prevenir condiciones de carrera y deadlocks.
- Implementar políticas de fairness en el acceso a recursos.
- Generar logs detallados para análisis de comportamiento.

0.2. Tecnologías Utilizadas

- **Lenguaje:** Java 11+
- **Mecanismos de Sincronización:** Semaphores, Locks, Conditions, CyclicBarrier, Exchanger
- **Estructuras de Datos Concurrentes:** BlockingQueue, AtomicInteger, AtomicLong
- **Visualización:** HTML/JavaScript para análisis de logs

1. Arquitectura General del Sistema

1.1. Diagrama de Arquitectura de Alto Nivel

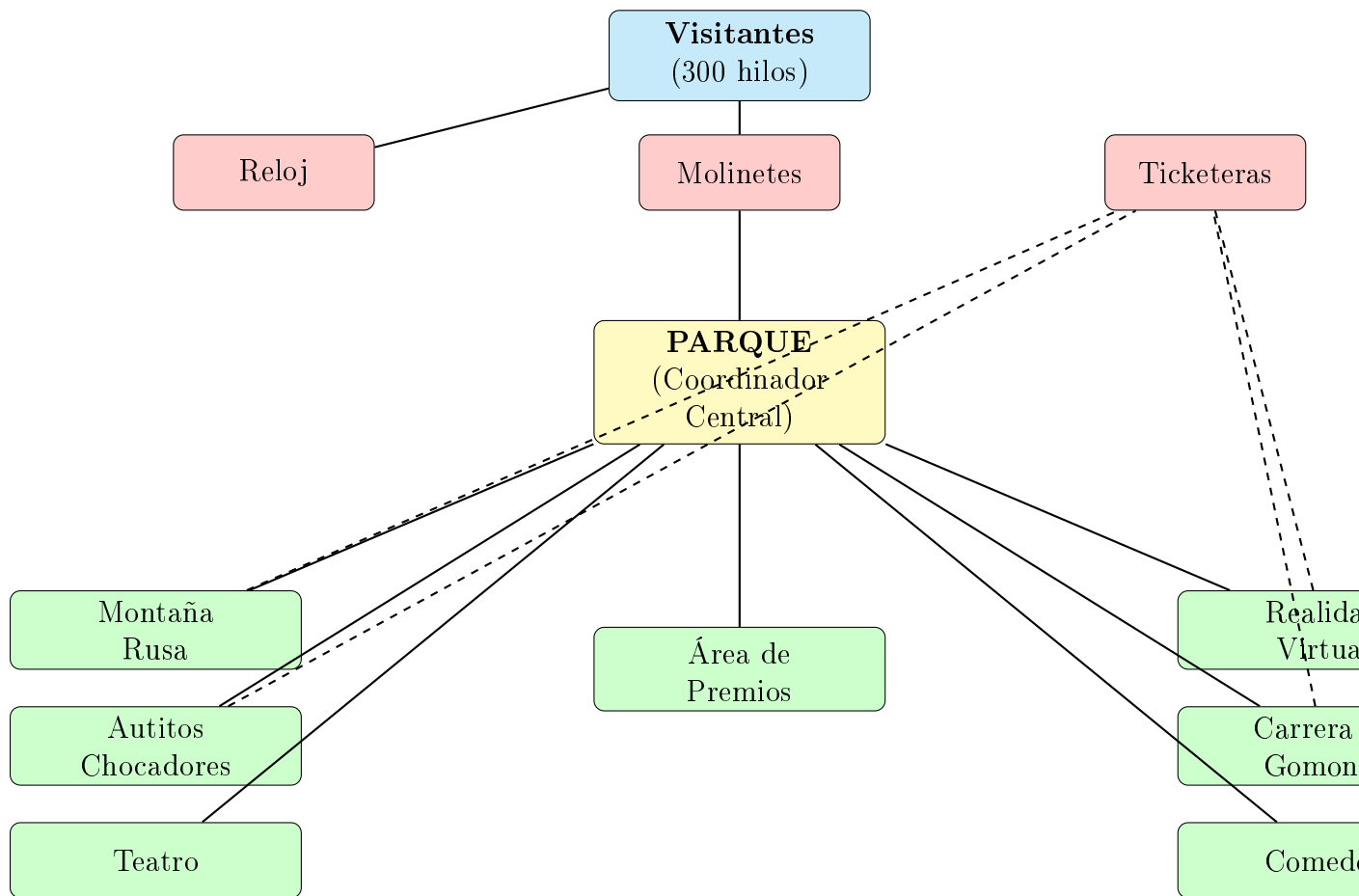


Figura 1: Arquitectura General del Sistema

1.2. Patrón Arquitectónico

El sistema implementa una **arquitectura basada en componentes** con los siguientes principios:

- **Alta cohesión:** Cada componente tiene una responsabilidad única y bien definida.
- **Bajo acoplamiento:** Los componentes interactúan a través de interfaces claras.
- **Separación de responsabilidades:** La lógica del dominio está separada de los mecanismos de sincronización.
- **Patrón Productor-Consumidor:** Utilizado en múltiples atracciones.
- **Patrón Coordinador:** El Parque actúa como coordinador central.

2. Componentes Principales

2.1. Main y Simulación

2.1.1. Main.java

Responsabilidad: Punto de entrada del sistema. Inicializa y configura la simulación.

Parámetros de configuración:

- APERTURA: Tiempo en milisegundos que el parque permanece abierto (12000 ms).
- CIERRE: Tiempo en milisegundos que el parque permanece cerrado (6000 ms).
- VISITANTES: Cantidad de hilos de visitantes a crear (300).
- MOLINETES: Cantidad de molinetes de entrada (3).

Flujo de ejecución:

1. Activa/desactiva generación de logs.
2. Inicia el hilo del Reloj.
3. Crea la instancia del Parque.
4. Genera los visitantes mediante Simulación.
5. Inicia todos los hilos de visitantes.

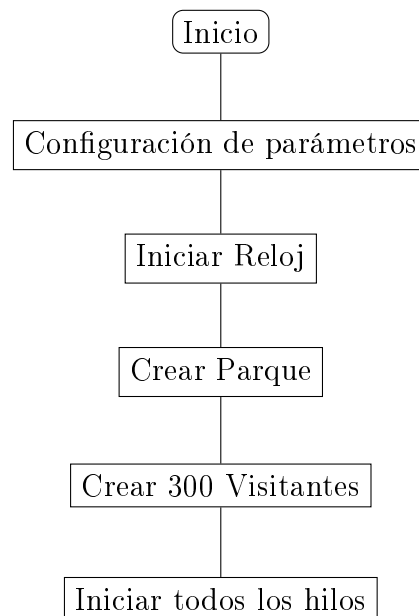


Figura 2: Flujo de inicialización del sistema

2.1.2. Simulacion.java

Responsabilidad: Fábrica para creación de visitantes con IDs únicos.

Características:

- Utiliza `AtomicLong` para generar IDs thread-safe.
- Crea instancias de `Visitante` con `BigInteger` como identificador.
- Retorna lista de hilos configurados pero no iniciados.

Mecanismo de sincronización:

```
1 private static final AtomicLong idGenerator = new AtomicLong(0);  
2 long uniqueId = idGenerator.incrementAndGet();
```

2.2. Sistema de Control Temporal

2.2.1. Reloj.java

Responsabilidad: Gestionar ciclos de apertura/cierre del parque.

Diagrama de estados:

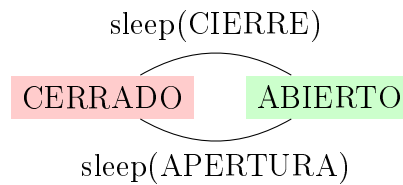


Figura 3: Estados del Reloj

Características técnicas:

- Variable estática `abierto`: Estado global accesible desde cualquier componente.
- Hilo daemon que corre continuamente.
- Método estático `operativo()`: Consulta no bloqueante del estado.
- Genera logs en cada cambio de estado.

Casos de uso:

- Molinetes verifican el estado antes de permitir ingreso.
- Visitantes consultan para saber si pueden continuar en atracciones.
- Atracciones pueden rechazar nuevos ingresos si el parque está cerrado.

2.3. Sistema de Ingreso

2.3.1. Molinete.java

Responsabilidad: Controlar ingreso de visitantes al parque con exclusión mutua.

Mecanismos de sincronización:

- **Semaphore(1, true):** Garantiza exclusión mutua con fairness.
- **BigInteger contador:** Generador de tickets protegido por la sección crítica.
- **Bloque finally:** Asegura liberación del semáforo incluso con excepciones.

Prevención de condiciones de carrera:

```
1 // Double-check del estado del parque
2 if (!Reloj.operativo()) return false;
3 try {
4     semaforo.acquire();
5     // Verificar nuevamente dentro de la sección crítica
6     if (!Reloj.operativo()) return false;
7     // ...
8 } finally {
9     semaforo.release();
10 }
```

2.3.2. Parque.java

Responsabilidad: Coordinador central y fachada del sistema.

Recursos gestionados:

- Lista de molinetes.
- Semáforo de entrada (entradaParque).
- Instancias de todas las atracciones.
- Generador de números aleatorios para decisiones probabilísticas.

Método mapa(): Bucle principal del visitante

```
1 while (Reloj.operativo()) {
2     if (rng(20)) montaniaRusa(v);
3     if (rng(20)) autitosChocadores(v);
4     if (rng(20)) comedor(v);
5     if (rng(20)) teatro(v);
6     if (rng(25)) carreraGomones(v);
7     // ... etc
8 }
```

Probabilidades de visita:

- 20 % - Montaña Rusa
- 20 % - Autitos Chocadores
- 20 % - Comedor
- 20 % - Teatro

- 20 % - Realidad Virtual
- 10 % - Área de Premios (requiere fichas)
- 25 % - Carrera de Gómones

2.4. Visitante y Sistema de Fichas

2.4.1. Visitante.java

Responsabilidad: Representar un agente concurrente que navega por el parque.

Atributos:

- ID: BigInteger - Identificador único inmutable.
- ticketNumero: BigInteger - Ticket asignado por el molinete.
- parque: Referencia al coordinador central.
- billetera: Sistema de fichas del visitante.

2.4.2. Billetera.java

Responsabilidad: Gestionar fichas del visitante.

Operaciones:

- cargarFichas(int): Incrementa fichas (acceso controlado por el caller).
- quitarFichas(int): Decrementa si hay suficientes.
- getFichas(): Consulta saldo actual.

Nota importante: La sincronización no está en esta clase, sino en los puntos de intercambio (Ticketera y Área de Premios).

2.4.3. Ticketera.java

Responsabilidad: Hilo que intercambia billeteras para cargar/descontar fichas.

Mecanismo: Exchanger Pattern

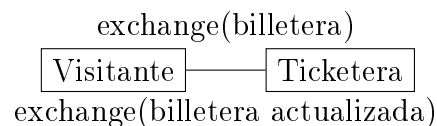


Figura 4: Intercambio sincronizado con Exchanger

Tipos de fichas:

- MR-FICHAS: 3 fichas (Montaña Rusa)
- AC-FICHAS: 2 fichas (Autitos Chocadores)
- AI-FICHAS: 1 ficha (Genérico)
- RV-FICHAS, GOMONES-FICHAS: valores por defecto

Parámetros:

- suma: true = entregar fichas, false = descontar.
- monto: cantidad fija (-1 para usar valorFicha()).

3. Atracciones Mecánicas

3.1. Montaña Rusa

Archivo: parque/juegosMecanicos/MontaniaRusa.java

Responsabilidad: Atracción con capacidad limitada, cola de espera y sistema de viajes por tandas.

Parámetros de configuración:

- CAPACIDAD_COLA: 10 visitantes.
- CAPACIDAD_VIAJE: 5 visitantes por carrito.

Mecanismos de sincronización:

Mecanismo	Capacidad	Propósito
BlockingQueue	10	Cola de espera FIFO con bloqueo
Semaphore (carrito)	5, fair	Control de acceso al carrito
Semaphore (barrier)	0	Barrera para sincronizar inicio de viaje
Semaphore (mutex)	1, fair	Exclusión mutua en obtención de fichas
Exchanger	-	Intercambio de billetera con Ticketera

Cuadro 1: Mecanismos de sincronización en Montaña Rusa

Algoritmo de barrera manual:

```

1 if (carrito.availablePermits() > 0) {
2     // No soy el ltimo , espero
3     barrier.acquire();
4 } else {
5     // Soy el ltimo , hago el viaje
6     Thread.sleep(5000);
7     // Despierto a los dem s
8     barrier.release(CAPACIDAD_VIAJE - 1);
9     // Reinicio el carrito
10    carrito.release(CAPACIDAD_VIAJE);
11 }
```

3.2. Autitos Chocadores

Archivo: parque/juegosMecanicos/AutitosChocadores.java

Responsabilidad: Atracción con capacidad total fija y sincronización mediante CyclicBarrier.

Configuración:

- PERSONAS_POR_AUTO: 2.
- Cantidad de autos: configurable al crear la instancia.
- CAPACIDAD_TOTAL: PERSONAS_POR_AUTO * cantidadAutos.

Mecanismos de sincronización:

- Semaphore autosDisponibles: Control de capacidad.

- **CyclicBarrier**: Sincroniza inicio cuando todos los lugares están ocupados.
- **Semaphore mutex(1)**: Protege intercambio con Ticketera.

4. Atracciones Complejas

4.1. Carrera de Gomones

Responsabilidad: Atracción multi-etapa con múltiples recursos y coordinación compleja.

Componentes del sistema:

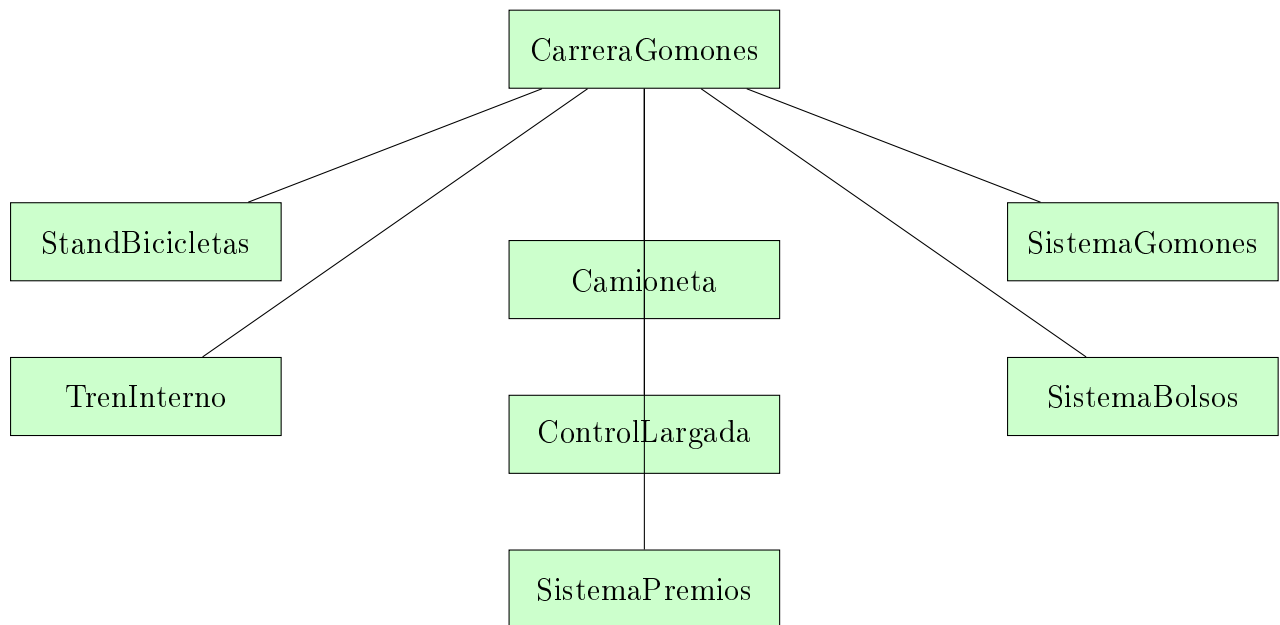


Figura 5: Arquitectura de Carrera de Gomones

4.1.1. Flujo Completo de Participación

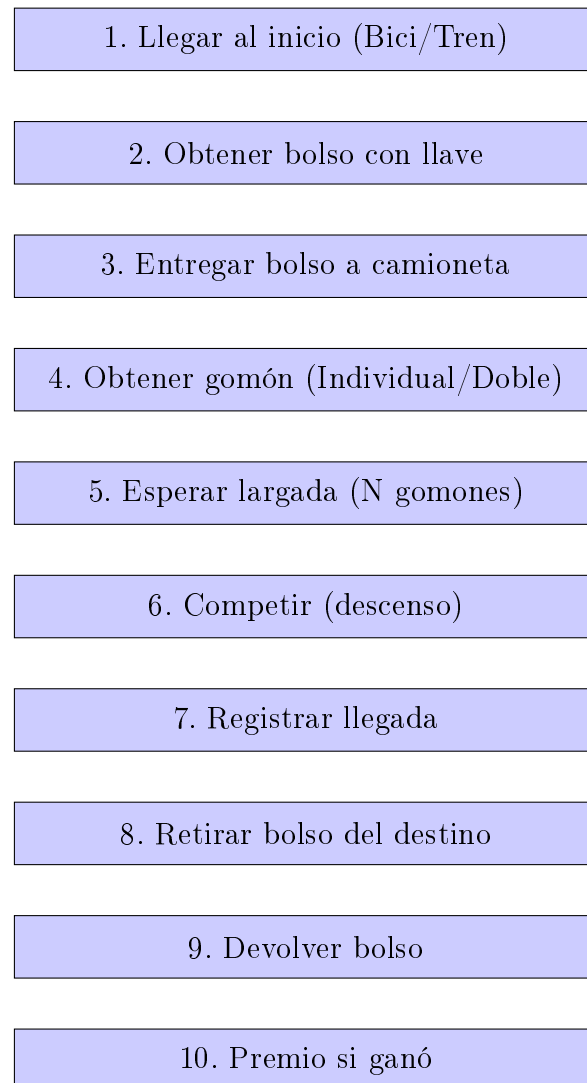


Figura 6: Flujo de participación en la Carrera de Gomones

5. Área de Premios: Diseño y Concurrencia

5.1. Responsabilidad del Componente

El `AreaPremios` funciona como el punto central de canje de fichas por regalos. Es un **recurso compartido y crítico** que gestiona el catálogo de premios y una cola de solicitudes. Su diseño se enfoca en la **concurrencia segura** utilizando el patrón **Productor-Consumidor** de Java.

5.2. Diagrama de Clases Simplificado (UML)

El diseño se compone de tres clases principales que interactúan para gestionar la solicitud de premios, más una clase interna para el control de sincronización.

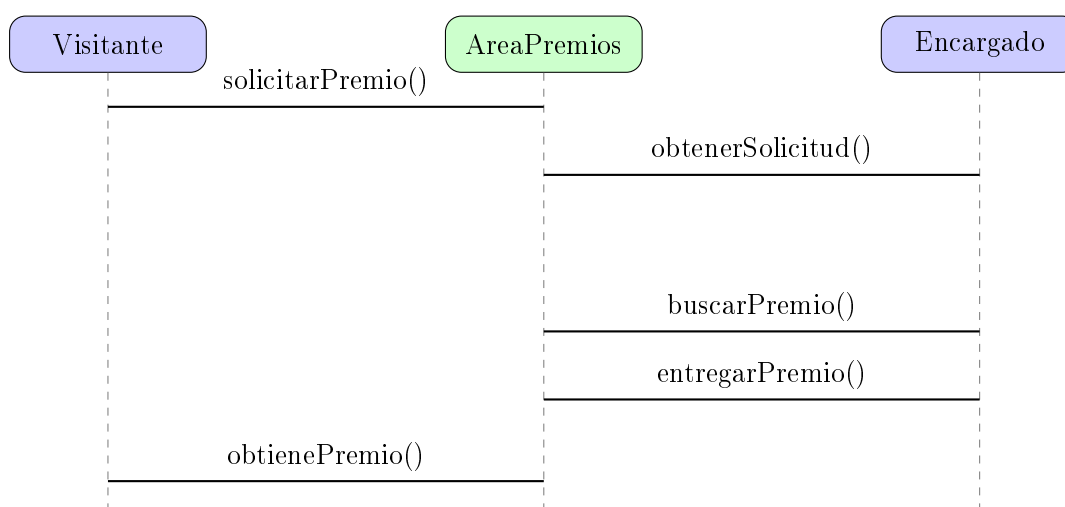


Figura 7: Diagrama de Interacción Productor-Recurso-Consumidor

5.3. Catálogo Inicial de Premios

La lógica de dominio del encargado busca siempre el premio de **mayor costo** que el visitante pueda pagar y que **tenga stock** (Política de Priorización de Gasto).

Cuadro 2: Catálogo de Premios Inicial

Nombre del Premio	Costo (Fichas)	Stock Inicial
Llavero	1	50
Sticker	1	100
Gorra	3	30
Remera	5	25
Peluche pequeño	8	20
Taza	10	15
Mochila	12	10
Peluche grande	15	8
Auriculares	20	5
Reloj	25	3
Consola portátil	30	2

5.4. Modelo de Concurrencia y Sincronización

El `AreaPremios` implementa el patrón ****Productor-Consumidor**** utilizando `ReentrantLock` y `Conditions`.

1. `ReentrantLock`: Provee **exclusión mutua** con fairness.
2. `Condition hayVisitantes`: Condición global para el **Encargado** (consumidor).
3. `Condition miCondition`: Condición **única por Visitante** (productor), usada para el despertar preciso.

5.5. Lógica de programa

Método `AreaPremios.buscarMejorPremio` Este método implementa la ****Política de Priorización de Gasto****, asegurando que el visitante siempre obtenga el premio de mayor valor que pueda pagar, si hay stock.

```
1 Premio buscarMejorPremio(int fichasDisponibles) {
2     lock.lock();
3     try {
4         Premio mejorPremio = null;
5         // 1. Itera sobre el cat logo
6         for (Premio premio : catalogoPremios) {
7             // 2. Verifica Stock y Costo
8             if (premio.hayStock() && premio.getCostoFichas() <=
9                 fichasDisponibles) {
10                 // 3. Prioriza el MAYOR costo
11                 if (mejorPremio == null ||
12                     premio.getCostoFichas() > mejorPremio.getCostoFichas
13                     ()) {
14                     mejorPremio = premio;
15                 }
16             }
17         }
18         return mejorPremio;
19     } finally {
20         lock.unlock();
21     }
22 }
```

Listing 1: `AreaPremios.buscarMejorPremio` - Lógica de Priorización

6. Comedor

6.1. Descripción General

El comedor contiene un conjunto fijo de mesas. Cada visitante que desea almorzar solicita una mesa libre; si no hay mesas disponibles, el visitante espera. Una vez que una mesa alcanza su capacidad máxima, todos los visitantes de la mesa comienzan a comer. Cuando todos terminan, la mesa vuelve a quedar disponible y el comedor despierta a los visitantes que estaban esperando lugar.

El sistema garantiza:

- Exclusión mutua en el acceso a las mesas.
- Evitar deadlocks gracias al orden estricto de adquisición de locks.
- Despertar eficiente de hilos bloqueados.

6.2. Clases Principales

Comedor

- Mantiene la lista de mesas.
- Gestiona el lock global que controla el acceso a la búsqueda de mesas disponibles.
- Contiene la condición `hayLugar`, usada para suspender visitantes cuando no existen mesas disponibles.
- Orquesta todo el flujo de almuerzo.

Mesa

- Cada mesa tiene su propio lock para gestionar a sus visitantes.
- Usa la condición `mesaCompleta` para bloquear a los visitantes hasta que la mesa llegue a capacidad máxima.
- Controla el estado interno: visitantes sentados, bandera de “comiendo”, y notificación al comedor cuando la mesa queda libre.

Flujo de interacción entre visitante, comedor y mesa

6.3. Flujo de Ejecución Detallado

1. Llegada del visitante

1. El visitante llama a `almorzar()`.
2. El comedor intenta asignarle una mesa.

2. Búsqueda de una mesa disponible

- El comedor toma su lock global.
- Itera sobre todas las mesas.
- Si una mesa está disponible y acepta al visitante con `intentarseSentarse()`, el visitante es asignado.
- Si no existe ninguna mesa disponible, el visitante espera en la condición `hayLugar`.

3. Espera hasta completar la mesa

- Cada visitante llama `esperarYComer()` en su mesa asignada.
- Se bloquea en `mesaCompleta.await()` hasta que la mesa alcance su capacidad.

4. Inicio del almuerzo

- El primer visitante que detecta que la mesa está completa activa el estado `comiendo = true`.
- Se despierta a todos los visitantes de la mesa.

5. Finalización y liberación

- Cada visitante termina de comer y decrementa el contador de ocupación de la mesa.
- El último en abandonar la mesa la marca como libre.
- Se notifica al comedor mediante `notificarLugarDisponible()` para despertar visitantes en espera.

6.4. Mecanismos de Concurrency Utilizados

Locks y Bloqueos

- Se emplea un `ReentrantLock` por mesa.
- Existe un lock global en el comedor para controlar asignaciones.

Condiciones

- `hayLugar`: usada por visitantes cuando no hay mesas disponibles.
- `mesaCompleta`: usada por las mesas para esperar a que la capacidad se complete.

Prevención de Deadlocks

- Nunca se toma el lock de `Comedor` desde dentro del lock de una `Mesa`.
- La notificación al comedor siempre se hace **fuera** del bloqueo de la mesa.

7. Teatro

7.1. Descripción General

El módulo **Teatro** modela un sistema concurrente donde múltiples visitantes intentan acceder a una sala y varios asistentes se encargan de formar grupos de ingreso. El teatro funciona por tandas: cada tanda consta de la formación de 4 grupos de 5 personas. Una vez formados los grupos, comienza el espectáculo; durante este tiempo nadie puede entrar.

Se emplean hilos tanto para los asistentes como para los visitantes, y se utiliza un mecanismo de exclusión mutua mediante **ReentrantLock** y varias condiciones para coordinar estados y puntos de sincronización.

7.2. Clases Principales

- **Teatro**: administra la cola de visitantes, los grupos, el inicio y fin del espectáculo, y coordina toda la sincronización.
- **Asistente**: hilo encargado de formar grupos siempre que haya suficientes visitantes y que el espectáculo no haya iniciado.
- **Visitante**: hilo que intenta entrar al teatro, espera ser habilitado por un asistente y participa del flujo completo del espectáculo.

7.3. Flujo del Sistema

7.3.1. Visitante

1. Intenta entrar; si hay espectáculo en curso, se rechaza.
2. Se encola en el lobby.
3. Espera a ser habilitado por un asistente.
4. Una vez habilitado, se sienta y espera el inicio del espectáculo.
5. Disfruta el espectáculo.
6. Espera el fin y se retira.

7.3.2. Asistente

1. Espera que haya visitantes en cola.
2. Forma grupos de 5 personas.
3. Cuando se formaron 4 grupos, inicia el espectáculo.
4. Tras 10 segundos, finaliza el espectáculo.

7.4. Mecanismos de Sincronización Utilizados

- **ReentrantLock**: asegura exclusión mutua en toda operación crítica del teatro.
- **Condition hayVisitantes**: usada por los asistentes para esperar a que haya suficientes visitantes en la cola.
- **Condition visitantesLiberados**: los visitantes esperan aquí hasta ser seleccionados para un grupo.
- **Condition inicioEspectaculo**: los visitantes esperan hasta que se dé inicio oficial.
- **Condition finEspectaculo**: los visitantes esperan la finalización para retirarse.

7.5. Estados del Teatro

- **esperando_visitantes**: no hay suficientes personas para formar grupo.
- **formando_grupos**: asistentes asignando grupos hasta llegar al máximo.
- **espectaculo_activo**: espectáculo en curso, nadie entra ni se forman grupos.
- **finalizado**: se limpian grupos y se permite una nueva tanda.

8. Realidad Virtual